

33 | 告警：如何搭建基于 HTTP 状态告警？

王炜 · 云原生架构与GitOps实战



你好，我是王炜。

上一节课，我们学习了如何借助 Ingress-Nginx Dashboard 快速搭建业务 HTTP 状态监控。它内置了多项 HTTP 请求指标，例如 TPS、请求成功率、P99 延迟等，这些指标对于我们掌握业务整体的可用性有非常大的帮助。

不过，在生产环境下，人工实时观察这些指标是不现实的，我们需要有一种能够自动发现异常指标的机制，它能在发现异常时发出通知。

这种机制也就是我们常说的**告警**。

这节课，我会继续深入可观测性，以 HTTP 请求成功率为例，带你从零搭建该指标的告警策略，并在这个过程中进一步讲解告警策略的配置方法，让你掌握给任意指标配置告警策略的能力。

在开始今天的学习之前，你需要按照 [第 32 讲](#) 的内容部署示例应用，并配置好 Prometheus 和 Ingress-Nginx Dashboard。

选择告警指标

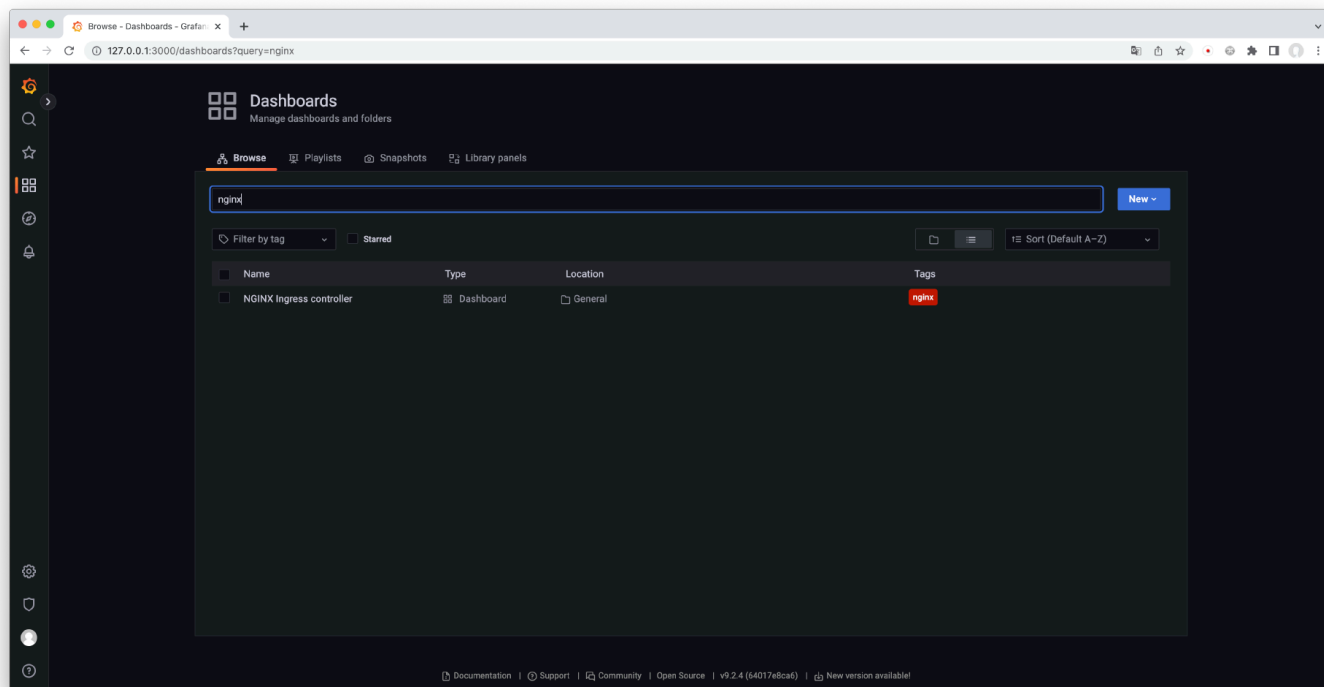
对于初学者来说，上手编写一段 PromQL 并不容易，为了降低实战门槛，我们直接从 Dashboard 中选择已有的指标配置告警策略。

首先，对 Grafana Service 进行端口转发。

 复制代码

```
1 $ kubectl port-forward --namespace prometheus service/prometheus-grafana 3000:80
```

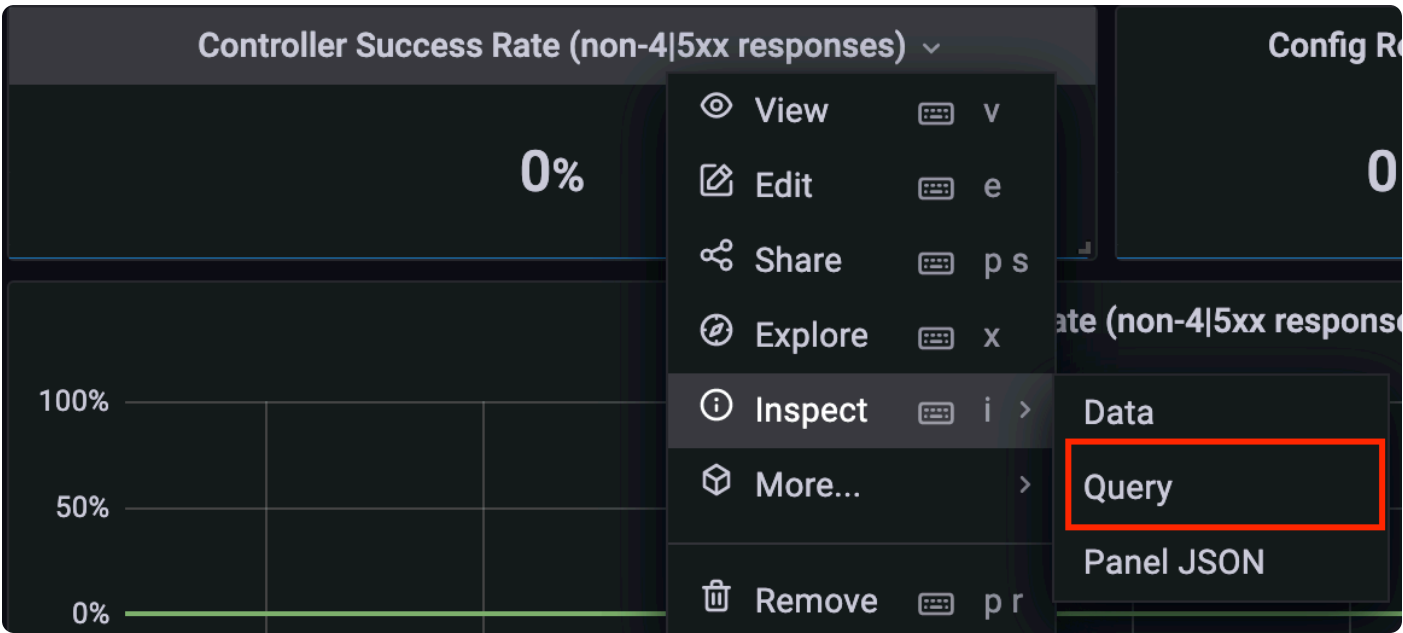
然后，使用浏览器访问 <http://127.0.0.1:3000> 打开 <http://127.0.0.1:3000>，使用账号密码 admin/prom-operator 登录，点击左侧的 Dashboards 界面，搜索“nginx”并进入 Ingress-Nginx Dashboard，如下图所示。



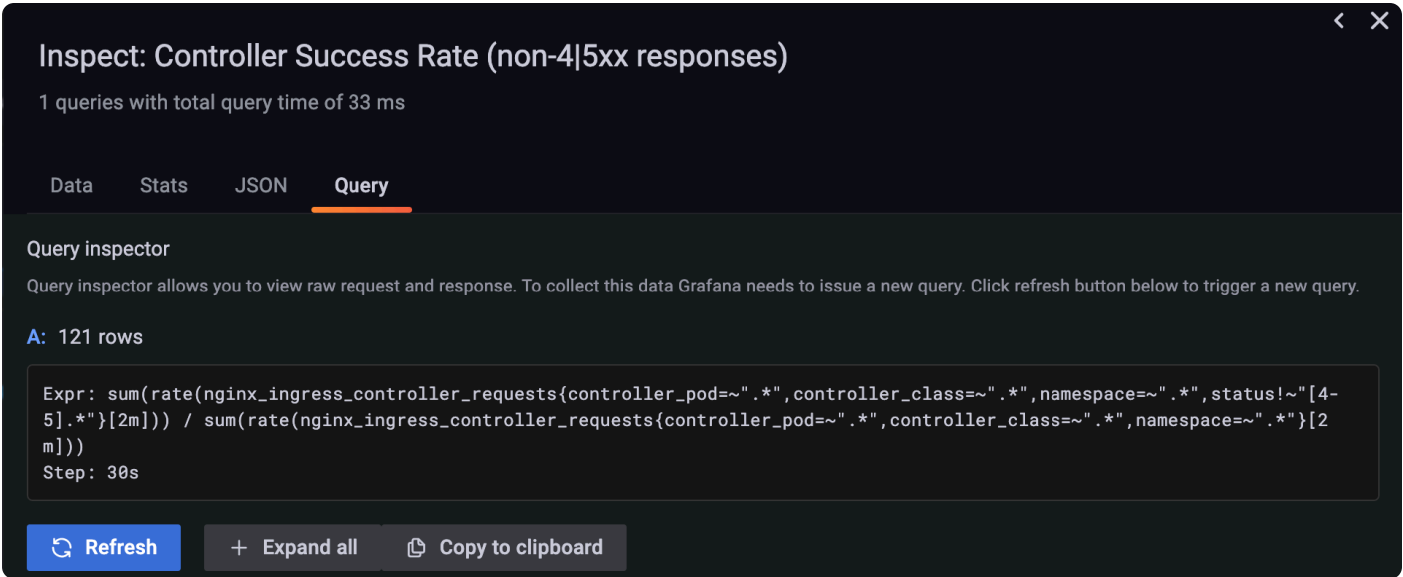
进入 Ingress-Nginx Dashboard 之后，会看到界面中展示了丰富的 HTTP 请求指标。在生产环境下，我们一般会非常关注业务 HTTP 请求的总体成功情况，所以，这里我们选择反映接口的请求成功率的 Controller Success Rate 指标，并为它配置告警策略。

选择好指标后，接下来我们需要**获取该指标的 PromQL 语句**进一步配置告警策略。

点击 Controller Success Rate 指标的标题展开操作菜单，并选择"Inspect"->"Query"查看指标 PromQL 语句。



在弹出的界面中，Expr 字段实际上就是这个指标的查询语句。



这条 PromQL 查询语句相对比较复杂，如果你现在还不能理解也没关系，我们现阶段只需要知道它反映了 HTTP 接口的总体可用性就可以了。接下来，我们将 Expr 字段的内容复制下来，方便在配置告警策略时使用。

配置告警策略

接下来，我们开始为 HTTP 请求成功率指标配置告警策略。

要配置告警策略，你需要创建 PrometheusRule CRD 对象，将下面的内容保存为 rule.yaml 文件。

 复制代码

```
1 apiVersion: monitoring.coreos.com/v1
2 kind: PrometheusRule
3 metadata:
4   labels:
5     release: prometheus
6     name: http-success-rate
7     namespace: prometheus
8 spec:
9   groups:
10  - name: nginx.http.rate
11    rules:
12  - expr: |
13      sum(rate(nginx_ingress_controller_requests{controller_pod=~".*",controller
14      for: 1m
15      alert: HTTPSuccessRateDown
16      annotations:
17        summary: "HTTP 请求成功率小于 90%"
18        description: "HTTP 请求成功率小于 90%，请及时处理"
```

在这段内容里，我们重点关注 labels、expr、for 和 annotations 字段。

labels 字段需要匹配 Prometheus CRD 对象中的 ruleSelector 配置，你可以通过下面的命令来查看 Prometheus CRD 的 ruleSelector 配置。

 复制代码

```
1 $ kubectl get Prometheus prometheus-kube-prometheus-prometheus -n prometheus -o j
```

```
2 {"matchLabels":{"release":"prometheus"}}
```

如果你是通过其他方式部署的 Prometheus，那么一定要确保 PrometheusRule 的 labels 和 Prometheus CRD 的 ruleSelector 配置一致，否则 PrometheusRule 可能不会生效。

expr 字段用来配置告警策略，它实际上是一个断言语句，它代表当 HTTP 请求成功率小于等于 90% 的时候，就发出告警。

for 字段表示 expr 表达式会在持续多久之后发出告警，例如持续 1 分钟满足条件则发出告警。

annotations 字段用来配置发出的告警信息，除了固定的告警信息以外，这里还可以使用 Go template 表达式来访问 Prometheus 内置的 labels 来实现动态的告警内容，例如 `{{ $labels.instance }}`、`{{ $labels.job }}` 和 `{{ $labels.pod }}` 等，详细信息你可以参考这份 [文档](#)。

最后，将 PrometheusRule 应用到集群内。

 复制代码

```
1 $ kubectl apply -f rule.yaml
2 prometheusrule.monitoring.coreos.com/http-success-rate created
```

检查 PrometheusRule 是否生效

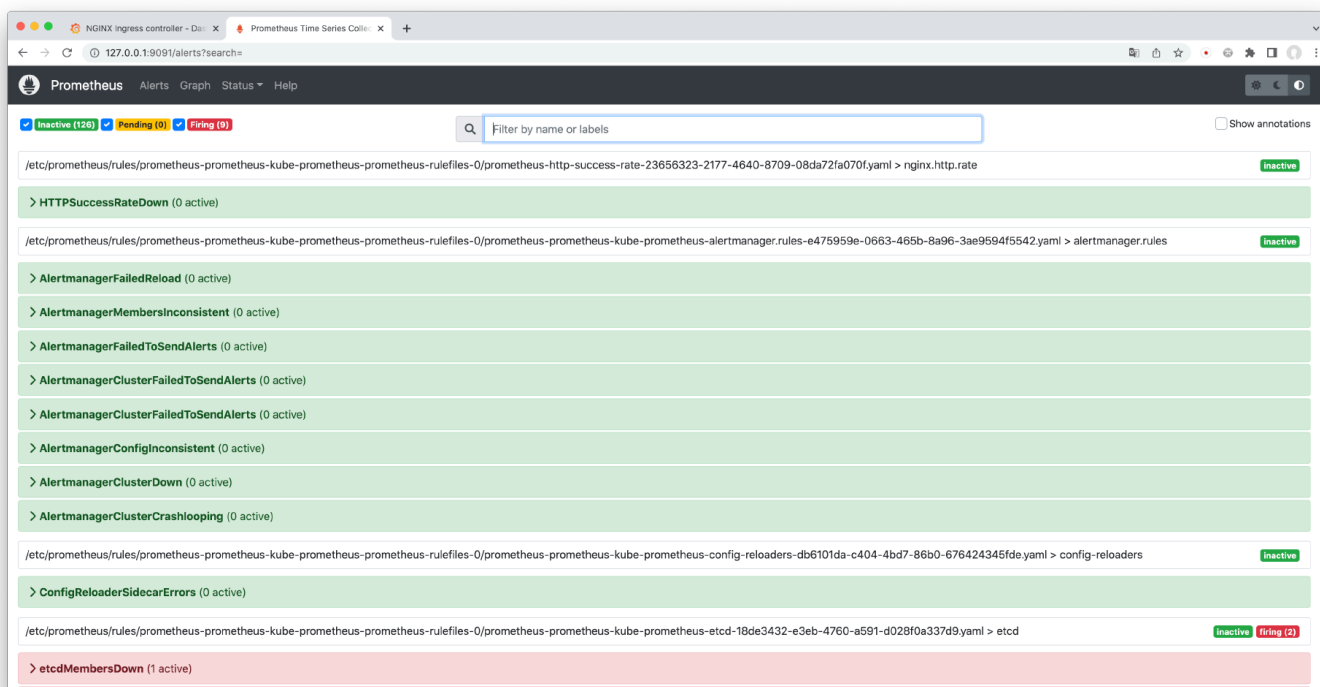
在将 PrometheusRule CRD 应用到集群之后，接下来我们需要检查配置是否生效。你可以进入 Prometheus 控制台进行查看。

在访问控制台之前，首先需要对 Prometheus Service 进行端口转发。

复制代码

```
1 $ kubectl port-forward --namespace prometheus service/prometheus-kube-prometheus-
```

接下来，打开浏览器，访问 <http://127.0.0.1:9001>，你应该能看到 Prometheus 控制台界面。点击菜单栏的 Alert 进入告警页面。



在 Alert 页面中，如果看到我们刚才配置的 HTTPSuccessRateDown，右侧的状态为 **Inactive**，说明告警配置成功了，告警状态为**未激活**，说明是正常状态。你可以进一步展开并查看详情。



从详情页面中我们可以看到告警策略的语句和描述信息。

到这里，我们已经成功配置了 HTTP 请求成功率的告警策略。

配置通知

配置完告警策略之后，我们还需要配置告警通知，这样当告警生效时，Prometheus 才会将告警信息发送给我们。

Prometheus 通知的拓展性非常强，你可以将通知发送到常见的渠道里，例如企业微信、钉钉和飞书等。不过，我想先向你介绍适用性最强且配置相对简单的邮箱通知。

获取邮箱 SMTP 密码

要让 Prometheus 以邮件的方式发送通知，首先需要提供邮箱的 SMTP 信息，以便 Prometheus 调用邮箱服务器发送邮件。

以 QQ 邮箱为例，首先我们进入设置页面，然后进入设置页面中的“账户”菜单，如下图所示。



然后，在“账户”菜单中找到 SMTP 设置，开启 SMTP 并生成授权码。

POP3/IMAP/SMTP/Exchange/CardDAV/CalDAV服务

开启服务：	POP3/SMTP服务 (如何使用 Foxmail 等软件收发邮件?)	已开启 关闭
	IMAP/SMTP服务 (什么是 IMAP, 它又是如何设置?)	已关闭 开启
	Exchange服务 (什么是Exchange, 它又是如何设置?)	已关闭 开启
	CardDAV/CalDAV服务 (什么是CardDAV/CalDAV, 它又是如何设置?)	已关闭 开启
(POP3/IMAP/SMTP/CardDAV/CalDAV服务均支持SSL连接。如何设置?)		

温馨提示：在第三方登录QQ邮箱，可能存在邮件泄露风险，甚至危害Apple ID安全，建议使用QQ邮箱手机版登录。
继续获取授权码登录第三方客户端邮箱  **生成授权码**



在生成授权码后，你将得到一串密码，我们把它复制并保存下来。

生成授权码



在第三方客户端登录时，密码框请输入以下授权码：

wzof 

提示：你可拥有多个授权码，所以无需记住该授权码，也不要告诉其他人。[了解更多](#)

确定

配置 Prometheus 发信邮箱

获得邮箱的 SMTP 密码之后，我们需要配置 Prometheus SMTP 发信邮箱。

你需要将下面的内容保存为 alertmanager.yaml 文件。

```
1 global:
```

 复制代码

```
2  resolve_timeout: 5m
3  smtp_from: 邮箱账户
4  smtp_auth_username: 邮箱账户
5  smtp_auth_password: 授权码
6  smtp_require_tls: false
7  smtp_smarthost: 'smtp.qq.com:465'
8
9  route:
10   receiver: 'email-alert'
11   group_by: ['job']
12
13   routes:
14   - receiver: 'email-alert'
15     group_wait: 50s
16     group_interval: 5m
17     repeat_interval: 12h
18
19  receivers:
20  - name: email-alert
21    email_configs:
22    - to: 接收邮箱
```

注意，你需要把上面的 `smtp_from`、`smtp_auth_username`、`smtp_auth_password` 和 `email_configs.to` 修改为你实际的邮箱信息。为了方便测试发送和接收邮件，你可以将 `smtp_from`、`smtp_auth_username` 和 `email_configs.to` 字段配置都配置为同一个邮箱账户。

当告警被触发时，为了避免频繁接收到通知，我们配置了 `repeat_interval` 字段，它表示在 12 小时之内只发送一次相同的告警信息。

此外，`receivers.email_configs` 字段也可以配置多个接收邮箱。

接下来，将 `alertmanager.yaml` 文件内容进行 Base64 编码。

 复制代码

```
1 $ cat alertmanager.yaml | base64
2 Z2xvYmFsOgogIHJlc29sdmVfdGltZW91dDogNW0KICBzbXRwX2Zyb206IGhhaW1pYW5ndW1hQGZveG1ha
```

然后，使用 `kubectl edit` 命令编辑 Prometheus Secret。

 复制代码

```
1 $ kubectl edit secret -n prometheus alertmanager-prometheus-kube-prometheus-alert
2
3 # Please edit the object below. Lines beginning with a '#' will be ignored,
4 # and an empty file will abort the edit. If an error occurs while saving this fil
5 # reopened with the relevant failures.
6 #
7 apiVersion: v1
8 data:
9   alertmanager.yaml: Z2xvYmFsOgogIHJlc29sdmVfdGltZW91dDogNW0.... # 替换内容
10 kind: Secret
```

然后将 `alertmanager.yaml` 字段后面的内容替换为上一步生成的 Base64 字符串，保存修改。

接下来，你还需要进入 Alertmanager 控制台检查配置是否生效。

在访问 Alertmanager 控制台之前，你需要先进行端口转发操作。

 复制代码

```
1 $ kubectl port-forward svc/prometheus-kube-prometheus-alertmanager -n prometheus
```

然后，使用浏览器打开 <http://127.0.0.1:9093/#/status>，检查配置信息是否生效。

Config

```
global:
  resolve_timeout: 5m
  http_config:
    follow_redirects: true
  smtp_from: haimianguma@foxmail.com
  smtp_hello: localhost
  smtp_smarthost: smtp.qq.com:465
  smtp_auth_username: haimianguma@foxmail.com
  smtp_auth_password: <secret>
  smtp_require_tls: false
  pagerduty_url: https://events.pagerduty.com/v2/enqueue
  opsgenie_api_url: https://api.opsgenie.com/
  wechat_api_url: https://qyapi.weixin.qq.com/cgi-bin/
  victorops_api_url: https://alert.victorops.com/integrations/generic/20131114/alert/
  telegram_api_url: https://api.telegram.org
route:
  receiver: email-alert
  group_by:
  - job
  continue: false
  routes:
  - receiver: email-alert
    continue: false
    group_wait: 50s
    group_interval: 5m
    repeat_interval: 12h
receivers:
- name: email-alert
  email_configs:
  - send_resolved: false
    to: haimianguma@foxmail.com
    from: haimianguma@foxmail.com
    hello: localhost
    smarthost: smtp.qq.com:465
    auth_username: haimianguma@foxmail.com
    auth_password: <secret>
    headers:
      From: haimianguma@foxmail.com
      Subject: '{{ template "email.default.subject" . }}'
```

更新 Secret 可能需要几分钟后才能生效，如果你看到了刚才我们配置的邮箱信息，说明邮箱配置成功了。

触发告警

要触发告警，我们首先需要访问 [🔗 上一节课](#) 部署的示例应用，以便产生 HTTP 请求指标数据。

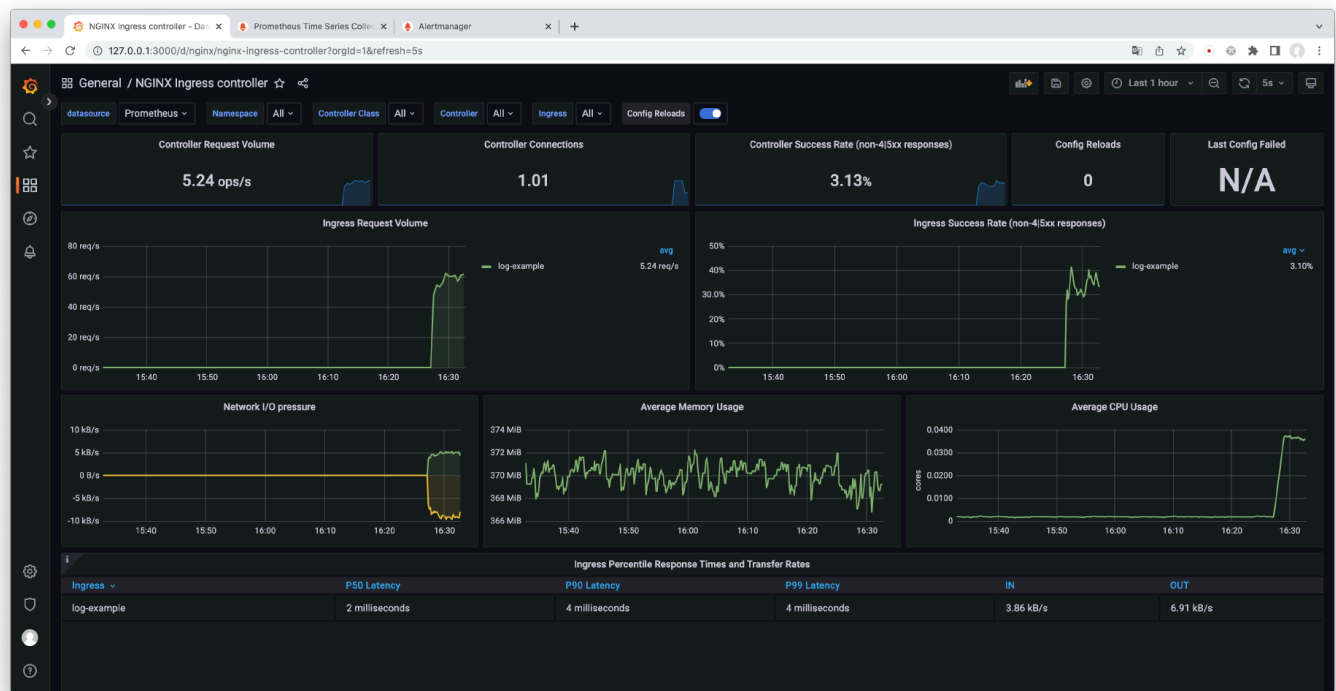
在示例应用中，我特意随机返回了 400 和 500 的状态码，所以 HTTP 请求成功率一定会小于 90%，这样我们就可以触发告警了。

接下来，使用下面的命令来访问示例应用。

复制代码

```
1 $ while true; do ; curl http://log-example.com/http ; echo -e '\n'$(date);done
```

现在，你可以进入 Grafana 控制台并打开 Ingress-Nginx Dashboard 查看 HTTP 指标信息，等待几分钟后，你将看到实时的指标。



从 Dashboard 中我们可以看到 Controller Success Rate 指标，此时请求成功率小于 90%，这符合我们的预期。

接下来，使用浏览器重新打开 <http://127.0.0.1:9091/alerts>，进入 Prometheus Alerts 界面，你将看到我们配置的告警策略处于 Pending 状态。

/etc/prometheus/rules/prometheus-prometheus-kube-prometheus-prometheus-rulefiles-0/prometheus-http-success-rate-23656323-2177-4640-8709-08da72fa070f.yaml > nginx.http.rate				pending (1)
HTTPSuccessRateDown (1 active)				
name: HTTPSuccessRateDown				
expr: sum(rate(nginx_ingress_controller_requests{controller_class=~".*",controller_pod=~".*",namespace=~".*",status!~"[4-5].*"}[2m])) / sum(rate(nginx_ingress_controller_requests{controller_class=~".*",controller_pod=~".*",namespace=~".*",status!~"[4-5].*"}[2m]))				
for: 1m				
annotations:				
description: HTTP 请求成功率小于 90%，请及时处理				
summary: HTTP 请求成功率小于 90%				
Labels	State	Active Since	Value	
alertname=HTTPSuccessRateDown	PENDING	2023-01-08T08:42:13.754325138Z	0	

这是因为 Prometheus 检测到了 HTTP 的请求成功率低于阈值，但由于告警的持续时间还没有达到预定义的时长，所以并不会发送通知。

等待 1 分钟后，告警状态从 Pending 变成了 Firing 状态，表示告警正在生效。

/etc/prometheus/rules/prometheus-prometheus-kube-prometheus-prometheus-rulefiles-0/prometheus-http-success-rate-23656323-2177-4640-8709-08da72fa070f.yaml > nginx.http.rate

▼ HTTPSuccessRateDown (1 active)

name: HTTPSuccessRateDown

expr: sum(rate(nginx_ingress_controller_requests{controller_class=~".*",controller_pod=~".*",namespace=~".*",status!~"[4-5].*"}[2m])) / sum(rate(nginx_ingress_controller_requests{controller_class=~".*",controller_pod=~".*",namespace=~".*",status!~"[4-5].*"}[2m]))

for: 1m

annotations:

description: HTTP 请求成功率小于 90%，请及时处理

summary: HTTP 请求成功率小于 90%

Labels	State	Active Since	Value
alertname=HTTPSuccessRateDown	FIRING	2023-01-08T08:27:43.754325138Z	32.08020050125313

此时，打开邮箱，你应该能看到 Prometheus 发送的告警通知邮件。

5 alerts for

View In Alertmanager

[5] Firing

Labels

alertname = HTTPSuccessRateDown

prometheus = prometheus/prometheus-kube-prometheus-prometheus

Annotations

description = HTTP 请求成功率小于 90%，请及时处理

summary = HTTP 请求成功率小于 90%

[Source](#)

至此，我们就完成了从创建告警策略、配置邮件通知、触发告警和接收邮件告警通知的全过程。

CPU 使用率告警

在生产环境下，除了配置 HTTP 指标告警策略以外，我还建议你配置 Pod CPU 使用率告警，这会帮助我们提前感知业务 Pod 是否正处于流量高峰，并提前做好应对措施。

你需要将下面的内容保存为 `cpu-rule.yaml` 文件。

 复制代码

```
1 apiVersion: monitoring.coreos.com/v1
2 kind: PrometheusRule
3 metadata:
4   labels:
5     release: prometheus
6   name: cpu-usage
7   namespace: prometheus
8 spec:
9   groups:
10  - name: cpu.usage
11    rules:
12    - expr: |
13        round( 100 * sum( rate(container_cpu_usage_seconds_total{container_name!=
14          for: 1m
15        alert: ContainerCPUUsage
16        annotations:
17          summary: "Pod {{ $labels.pod }} CPU 使用率超过限制值 80%"
18          description: "Namespace {{ $labels.namespace }}, Pod {{ $labels.pod }}, %
```

在这个告警策略中，我们用容器的 CPU 使用量 / 容器的资源 Limit 值来计算容器的 CPU 使用率，当这个值大于 80% 的时候，触发报警。

需要注意的是，要让这个告警策略生效，你需要为工作负载配置资源的 request 和 limit，在示例应用中，我已经提前配置好了。

annotations 字段和我们之前配置的 HTTP 请求成功率的通知消息有所不同，这里我们使用了 Go Template 模板，并读取了内置的 labels 变量。这样，当告警被触发的时候，Prometheus 会自动填充这些变量，我们可以在告警通知里面直接得到命名空间、Pod 名称、容器名以及当前 CPU 的使用率。

接下来，我们访问示例应用来触发告警策略。

复制代码

```
1 $ while true; do ; curl http://log-example.com/http ; echo -e '\n'$(date);done
```

1–2 分钟后，你可以重新打开 [Prometheus Dashboard](#)，并查看 ContainerCPUUsage 告警策略的状态，你应该能看到告警正处于 Pending 状态。

/etc/prometheus/rules/prometheus-prometheus-kube-prometheus-prometheus-rulefiles-0/prometheus-cpu-usage-439a7b05-6b4c-4ce6-8c91-6b48f53fd0a6.yaml > cpu.usage

pending (2)

▼ ContainerCPUUsage (2 active)

name: ContainerCPUUsage

expr: round(100 * sum by (pod, container_name, namespace) (rate(container_cpu_usage_seconds_total{container_name!="POD"}[1m])) / sum by (pod, container_name, namespace) (kube_pod_container_resource_limits{container_name!="f

for: 1m

annotations:

description: Namespace {{ \$labels.namespace }}, Pod {{ \$labels.pod }}, 容器 {{ \$labels.container_name }} CPU 使用率为 {{ \$value }}, LABELS = {{ \$labels }}

summary: Pod {{ \$labels.pod }} CPU 使用率超过限制值 80%

Labels	State	Active Since	Value
alertname=ContainerCPUUsage namespace=default pod=log-example-56895d96d7-pp4h8	PENDING	2023-01-08T14:06:36.951368604Z	86
alertname=ContainerCPUUsage namespace=default pod=log-example-56895d96d7-h9vct	PENDING	2023-01-08T14:06:36.951368604Z	104

展开告警详情，会显示当前正处于 CPU 使用率告警的 Pod 信息。

等待 1 分钟后，此时 Pending 状态将变成 Firing 状态，并将收到 CPU 使用率告警的邮件通知，如下图所示。

Labels

alertname = ContainerCPUUsage
namespace = default
pod = log-example-56895d96d7-pp4h8
prometheus = prometheus/prometheus-kube-prometheus-prometheus

Annotations

description = Namespace default, Pod log-example-56895d96d7-pp4h8, 容器 CPU 使用率为 111, LABELS = map[namespace:default pod:log-example-56895d96d7-pp4h8]
summary = Pod log-example-56895d96d7-pp4h8 CPU 使用率超过限制值 80%

[Source](#)

至此，容器 CPU 使用率告警策略就配置完成了。

其他告警策略集合

要从零编写 Prometheus 告警策略并不容易，这里，我为你介绍一个社区维护的 PrometheusRules [🔗 集合仓库](#)，你可以在这个仓库中找到大部分需要在生产环境中配置的告警策略。

此外，这个集合仓库还提供了 [🔗 官网](#)，你也可以在这里通过关键字来查找告警策略。

其他通知方式

除了邮件通知，我们还可以配置其他的通知方式，例如企业微信、钉钉或者飞书。不过，要使用 Prometheus 原生的扩展方式配置这些通知比较复杂，这里我介绍一种通过 PrometheusAlert 项目来配置通知的方式，它配置起来相对简单，并且支持丰富的第三方通知。

安装 PrometheusAlert

首先，我们需要安装 PrometheusAlert 项目，你可以使用下面的命令来安装。

 复制代码

```
1 $ kubectl create ns monitoring
2 $ kubectl apply -n monitoring -f https://ghproxy.com/https://raw.githubusercontent.com/prometheus/alertmanager/master/manifests/alertmanager-k8s.yaml
3 configmap/prometheus-alert-center-conf created
4 deployment.apps/prometheus-alert-center created
5 service/prometheus-alert-center created
```

然后，等待工作负载就绪。

 复制代码

```
1 $ kubectl wait --for=condition=Ready pods --all -n monitoring
2 pod/prometheus-alert-center-75b7b6465-26zvd condition met
```

配置 PrometheusAlert

安装完成后，PrometheusAlert 会默认开启钉钉和企业微信通知的开关，但并没有开启飞书开关。这里我以飞书通知为例介绍一下配置方法。

首先我们需要启用飞书通知开关。要启用飞书通知，你需要编辑 PrometheusAlert 存储配置的 configmap。

 复制代码

```
1 $ kubectl edit configmap prometheus-alert-center-conf -n monitoring
2
3 #是否开启飞书告警通道,可同时开始多个通道0为关闭,1为开启
4 open-feishu=0
```

找到 open-feishu 配置项，并将 0 修改为 1，保存配置。

然后，要让配置立即生效，你可以删除旧的 Pod。

 复制代码

```
1 $ kubectl delete pod -l app=prometheus-alert-center -n monitoring
2 pod "prometheus-alert-center-75b7b6465-26zvd" deleted
```

这样，飞书通知的开关也就开启了。

创建飞书机器人

下一步，我们需要创建飞书机器人。首先，你需要创建一个群聊，然后点击群聊“设置”，选择“群机器人”。

设置



我的通知群

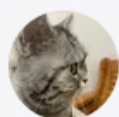
编辑群信息



群成员

1 >

Q 搜索



群机器人



在弹出的界面中，选择“添加机器人”，然后选择“自定义机器人”。



接下来，点击“添加”完成机器人创建过程，此时你将得到一个 webhook 地址，例如：

🔗 <https://open.feishu.cn/open-apis/bot/v2/hook/xxxx-xxx>，将它复制下来备用。

<>



第二步：配置 webhook

复制以下地址，在外部系统中完成 webhook 设置

webhook 地址

`https://open.feishu.cn/open-apis/bot/v2/hook/735c...`

复制

请保管好此 webhook 地址。不要公布在 Github、博客等可公开查阅的网站上。地址泄露后可能被恶意调用发送垃圾信息

安全设置

☐ 自定义关键词 ?

☐ IP 白名单 ?

☐ 签名校验 ?

设置

☐ 仅群主和添加者可编辑、移除此机器人

完成

配置 Alertmanager

接下来，我们还需要配置 Alertmanager，让所有通知都转发到 PrometheusAlert 去处理。

将下面的内容保存为 prometheusAlert.yaml 文件。

 复制代码

```
1 global:
2   resolve_timeout: 5m
3 route:
4   group_by: ['instance']
5   group_wait: 10m
6   group_interval: 1m
7   repeat_interval: 12h
8   receiver: 'web.hook.prometheusalert'
9 receivers:
10  - name: 'web.hook.prometheusalert'
11    webhook_configs:
```

```
12 - url: 'http://prometheus-alert-center.monitoring:8080/prometheusalert?type=fs&
```

从这段配置内容中我们会发现，这里本质上是使用了 Alertmanager 的 webhook 功能，将所有告警以 HTTP 的方式发送到了 PrometheusAlert，并由它来发送通知。

请注意，你需要将 url 字段中的 fsurl 参数替换为之前创建的飞书 webhook 地址。

当然你也可以使用其他的通知方式，例如企业微信或者钉钉，只需要构造不同的 url 参数就可以了，你可以参考 [这个链接](#)。

然后，将该文件内容进行 base64 编码。

 复制代码

```
1 $ cat prometheusAlert.yaml | base64
2 Z2xvYmFsOgogIHJlc29sdmVfdGltZW91dDogNW0Kcm91dGU6CiAgZ3JvdXBfYnk6IFsnaW5zdGFuY2UnX
```

接下来，编辑 Alertmanager 的 Secret 文件。

 复制代码

```
1 $ kubectl edit secret -n prometheus alertmanager-prometheus-kube-prometheus-alert
2
3 # Please edit the object below. Lines beginning with a '#' will be ignored,
4 # and an empty file will abort the edit. If an error occurs while saving this fil
5 # reopened with the relevant failures.
6 #
7 apiVersion: v1
8 data:
9   alertmanager.yaml: Z2xvYmFsOgogIHJlc29sdmVfdGltZW91dDogNW0.... # 替换内容
10 kind: Secret
11 .....
```

将 alertmanager.yaml 字段后面的内容替换为上一步生成的 Base64 字符串，保存修改。

为了让配置立即生效，你可以删除旧的 Alertmanager Pod。

 复制代码

```
1 $ kubectl delete pod -l app.kubernetes.io/name=alertmanager -n prometheus
2 pod "alertmanager-prometheus-kube-prometheus-alertmanager-0" deleted
```

触发告警

接下来，仍然使用下面的命令来访问示例应用，以便触发告警。

 复制代码

```
1 $ while true; do ; curl http://log-example.com/http ; echo -e '\n'$(date);done
```

等待几分钟后，你将在飞书收到告警通知。



自定义机器人 机器人 12:07

PrometheusAlert告警消息

Prometheus告警信息

ContainerCPUUsage

告警级别:

开始时间: 2023-01-09 04:06:18

结束时间: 0001-01-01 00:00:00

故障主机IP:

Namespace default, Pod log-example-56895d96d7-pp4h8, 容器 CPU 使用率为 106, LABELS = map[namespace:default pod:log-example-56895d96d7-pp4h8]

*** [点我屏蔽该告警](#)

PrometheusAlert告警消息

到这里，我们就完成了使用 PrometheusAlert 发送告警通知的全过程。

PrometheusAlert 支持的通知类型有非常多，除了常见的 IM，还支持短信和电话通知，你可以参考 [这个文档](#) 进行配置。

总结

这节课，我们学习了如何创建 Prometheus 告警策略，并通过配置邮件通知来接收告警。

对于新手来说，编写 PromQL 并不是一件容易的事，所以我教你了一个小技巧，也就是通过查看 Dashboard 的指标来获得查询语句，并将它配置成 PrometheusRule 对象，这样就完成

了指标告警配置。通过这种方式，理论上你可以对任何 Dashboard 中已有的指标配置告警策略。

在配置通知方面，我首先介绍了通用性最强也最简单的邮箱配置方法，不同的邮箱配置稍有差异，不过你只需要找到邮箱的 SMTP 服务器、账号和密码也就可以了。此外，我还介绍了 PrometheusAlert 项目，通过它你可以配置丰富的通知渠道，配置过程也相对比较简单。

最后，我还提到了社区维护的 PrometheusRules 集合仓库，如果你想找到更多现成的告警策略并深入学习 PromQL，那么从这里开始是一个不错的起点。

到这里，GitOps 的可观测性专题也就全部结束了。通过为业务系统配置日志、监控和告警策略，我相信你已经能全方位掌握业务的健康状态，也具备提前发现问题和解决业务隐患的能力了。

思考题

最后，给你留一道思考题吧。

你认为在生产环境下，我们还需要对哪些指标配置告警呢？说说你的理解。

欢迎你给我留言交流讨论，你也可以把这节课分享给更多的朋友一起阅读。我们下节课见。

AI智能总结

本文详细介绍了如何搭建基于 HTTP 状态告警系统的全过程，通过配置 Prometheus 和 Ingress-Nginx Dashboard 实现。读者可以学习到如何选择告警指标、配置告警策略和检查告警策略是否生效，以及如何配置告警通知和获取邮箱 SMTP 密码的步骤。文章还介绍了触发告警的步骤以及如何配置 Pod CPU 使用率告警。此外，还提到了其他通知方式的配置。通过配置 PrometheusAlert，读者可以实现丰富的通知渠道，配置过程相对简单。总的来说，本文内容涵盖了从安装到配置各种告警通知的全过程，对于需要监控和管理生产环境中的业务 HTTP 请求状态的读者具有实际指导意义。文章还提到了如何创建 Prometheus 告警策略，并通过配置邮件通知来接收告警，以及介绍了社区维护的 PrometheusRules 集合仓库。通过本文，读者可以全方位掌握业务的健康状态，具备提前发现问题和解决业务隐患的能力。

全部留言 (2)

最新 精选



夜空中最亮的星

2023-02-22 来自北京

讲解很详细



1



凡之

2024-05-13 来自北京

系统方面：cpu，内存（使用率），磁盘（io,使用率） 负载，网络（in/out）

服务信息：各种服务状态，进程，端口，信息，特定的信息，java服务gc，年轻待，老年代

中间件：通用指标：down，up，

redis: 使用的内存，客户端连接数等，’

nginx: 连接数，请求成功率，

MQ: 未消费消息数

mysql：慢日志

业务方面： api接口，业务错误日志

